# Background

The `Flexbox Layout` (Flexible Box) module (a W3C Candidate Recommendation as of October 2017) aims at providing a more efficient way to lay out, align and distribute space among items in a container, even when their size is unknown and/or dynamic (thus the word "flex").

The main idea behind the flex layout is to give the container the ability to alter its items' width/height (and order) to best fill the available space (mostly to accommodate to all kind of display devices and screen sizes). A flex container expands items to fill available free space or shrinks them to prevent overflow.

Most importantly, the flexbox layout is direction-agnostic as opposed to the regular layouts (block which is vertically-based and inline which is horizontally-based). While those work well for pages, they lack flexibility (no pun intended) to support large or complex applications (especially when it comes to orientation changing, resizing, stretching, shrinking, etc.).

**Note:** Flexbox layout is most appropriate to the components of an application, and small-scale layouts, while the Grid layout is intended for larger scale layouts.

## Background

O módulo Flexbox Layout (Flexible Box) (uma Recomendação Candidata do W3C em outubro de 2017) visa fornecer uma maneira mais eficiente de dispor, alinhar e distribuir espaço entre os itens em um contêiner, mesmo quando seu tamanho é desconhecido e/ou dinâmico ( daí a palavra “flex”).

A principal ideia por trás do layout flexível é dar ao container a capacidade de alterar a largura/altura de seus itens (e ordem) para melhor preencher o espaço disponível (principalmente para acomodar todos os tipos de dispositivos de exibição e tamanhos de tela). Um contêiner flexível expande itens para preencher o espaço livre disponível ou os reduz para evitar estouro.

Mais importante ainda, o layout flexbox é independente de direção, em oposição aos layouts regulares (bloco que é baseado na vertical e inline que é baseado na horizontal). Embora funcionem bem para páginas, eles não têm flexibilidade (sem trocadilhos) para suportar aplicativos grandes ou comp_lexos (especialmente quando se trata de

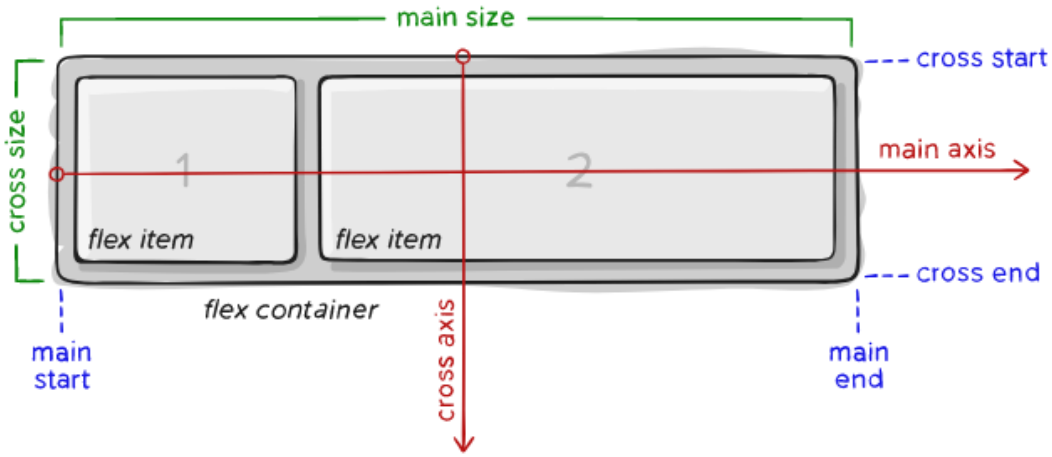
mudança de orientação, redimensionamento, alongamento, redução etc.).

Nota: O layout Flexbox é mais apropriado para os componentes de um aplicativo e layouts de pequena escala, enquanto o layout de Grade é destinado a layouts de escala maior.

## Basics and terminology

Since flexbox is a whole module and not a single property, it involves a lot of things including its whole set of properties. Some of them are meant to be set on the container (parent element, known as “flex container”) whereas the others are meant to be set on the children (said “flex items”).

If “regular” layout is based on both block and inline flow directions, the flex layout is based on “flex-flow directions”. Please have a look at this figure from the specification, explaining the main idea behind the flex layout.



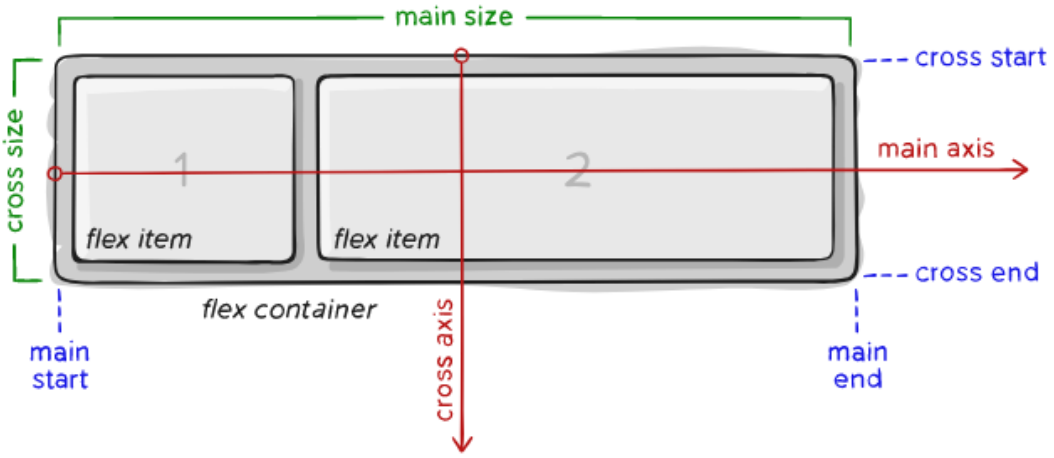

Items will be laid out following either the main axis (from main-start to main-end) or the cross axis (from cross-start to cross-end).

- main axis – The main axis of a flex container is the primary axis along which flex items are laid out. Beware, it is not necessarily horizontal; it depends on the flex-direction property (see below).
- main-start | main-end – The flex items are placed within the container starting from main-start and going to main-end.
- main size – A flex item's width or height, whichever is in the main dimension, is the item's main size. The flex item's main size property is either the 'width' or 'height' property, whichever is in the main dimension.
- cross axis – The axis perpendicular to the main axis is called the cross axis. Its direction depends on the main axis direction.
- cross-start | cross-end – Flex lines are filled with items and placed into the container starting on the cross-start side of the flex container and going toward the cross-end side.
- cross size – The width or height of a flex item, whichever is in the cross dimension, is the item's cross size. The cross size property is whichever of 'width' or 'height' that is in the cross dimension.

# Basics and terminology

Como o flexbox é um módulo inteiro e não uma propriedade única, ele envolve muitas coisas, incluindo todo o seu conjunto de propriedades. Alguns deles devem ser definidos no contêiner (elemento pai, conhecido como “contêiner flexível”), enquanto os outros devem ser definidos nos filhos (disse “itens flexíveis”).

Se o layout “regular” é baseado em direções de fluxo em bloco e em linha, o layout flexível é baseado em “direções de fluxo flexível”. Por favor, dê uma olhada nesta figura da especificação, explicando a ideia principal por trás do layout flexível.

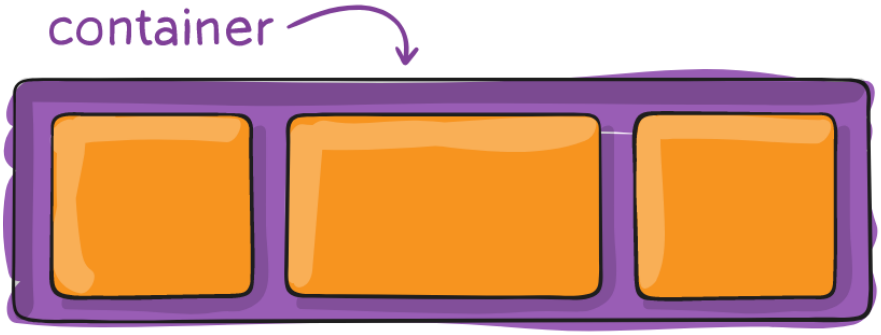


Os itens serão dispostos seguindo o eixo principal (do início ao fim principal) ou o eixo transversal (do início ao fim).

eixo principal – O eixo principal de um contêiner flexível é o eixo primário ao longo do qual os itens flexíveis são dispostos. Cuidado, não é necessariamente horizontal; depende da propriedade flex-direction (veja abaixo).
partida principal | main-end – Os itens flex são colocados dentro do container começando do main-start e indo para o main-end.
tamanho principal – a largura ou altura de um item flexível, o que estiver na dimensão principal, é o tamanho principal do item. A propriedade de tamanho principal do item flexível é a propriedade 'largura' ou 'altura', o que estiver na dimensão principal.
eixo transversal – O eixo perpendicular ao eixo principal é chamado de eixo cruzado. Sua direção depende da direção do eixo principal.
partida cruzada | cross-end – As linhas flexíveis são preenchidas com itens e colocadas no contêiner começando no lado de início cruzado do

contêiner flexível e indo em direção ao lado de extremidade cruzada.
tamanho cruzado – A largura ou altura de um item flexível, o que estiver na dimensão cruzada, é o tamanho cruzado do item. A propriedade de tamanho cruzado é qualquer uma de 'largura' ou 'altura' que esteja na dimensão cruzada.

## Flexbox properties




## Properties for the Parent (flex container)

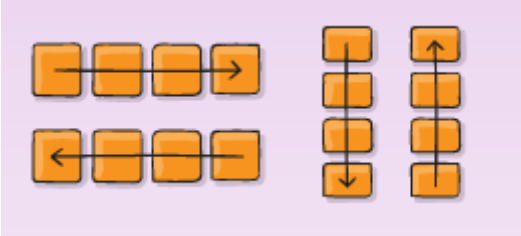
## display

This defines a flex container; inline or block depending on the given value. It enables a flex context for all its direct children.

```
.container {
  display: flex; /* or inline-flex */
}
```

Note that CSS columns have no effect on a flex container.

# flex-direction

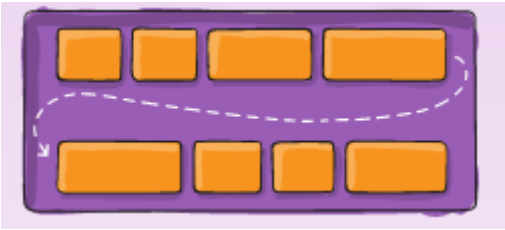


This establishes the main-axis, thus defining the direction flex items are placed in the flex container. Flexbox is (aside from optional wrapping) a single-direction layout concept. Think of flex items as primarily laying out either in horizontal rows or vertical columns.

```
.container {
  flex-direction: row | row-reverse | column | column-rever
}
```

- row (default): left to right in ltr; right to left in rtl
- row-reverse: right to left in ltr; left to right in rtl
- column: same as row but top to bottom
- column-reverse: same as row-reverse but bottom to top

# flex-wrap



By default, flex items will all try to fit onto one line. You can change that and allow the items to wrap as needed with this property.

```
.container {
  flex-wrap: nowrap | wrap | wrap-reverse;
}
```

nowrap (default): all flex items will be on one line

wrap: flex items will wrap onto multiple lines, from top to bottom.

wrap-reverse: flex items will wrap onto multiple lines from bottom to top.

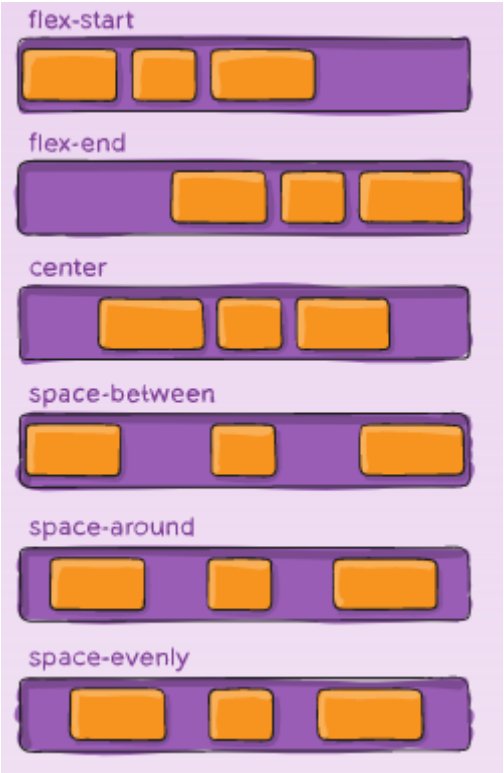
There are some visual demos of flex-wrap here.

# flex-flow

This is a shorthand for the flex-direction and flex-wrap properties, which together define the flex container's main and cross axes. The default value is row nowrap.

```
.container {
  flex-flow: column wrap;
}
```

# justify-content




This defines the alignment along the main axis. It helps distribute extra free space leftover when either all the flex items on a line are inflexible, or are flexible but have reached their maximum size. It also exerts some control over the alignment of items when they overflow the line.

```
.container {
  justify-content: flex-start | flex-end | center | space-b
}
```

flex-start (default): items are packed toward the start of the flex-direction.

flex-end: items are packed toward the end of the flex-direction.

start: items are packed toward the start of the writing-mode direction.

end: items are packed toward the end of the writing-mode direction.

left: items are packed toward left edge of the container, unless that doesn't make sense with the flex-direction, then it behaves like start.

right: items are packed toward right edge of the container, unless that doesn't make sense with the flex-direction, then it behaves like start.

center: items are centered along the line

space-between: items are evenly distributed in the line; first item is on the start line, last item on the end line

space-around: items are evenly distributed in the line with equal space around them. Note that visually the spaces aren't equal, since all the items have equal space on both sides. The first item will have one unit of space against the container edge, but two units of space between the next item because that next item has its own spacing that applies.

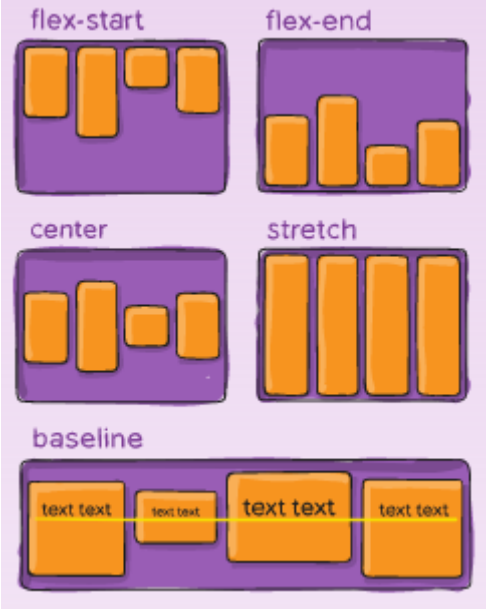
space-evenly: items are distributed so that the spacing between any two items (and the space to the edges) is equal.

Note that that browser support for these values is nuanced. For example, space-between never got support from some versions of Edge, and start/end/left/right aren't in Chrome yet. MDN has detailed charts. The safest values are flex-start, flex-end, and center.

There are also two additional keywords you can pair with these values: safe and unsafe. Using safe ensures that however you do

this type of positioning, you can't push an element such that it renders off-screen (e.g. off the top) in such a way the content can't be scrolled too (called "data loss").

# align-items




This defines the default behavior for how flex items are laid out along the **cross axis** on the current line. Think of it as the `justify-content` version for the cross-axis (perpendicular to the main-axis).

```
.container {
  align-items: stretch | flex-start | flex-end | center | ba
}
```

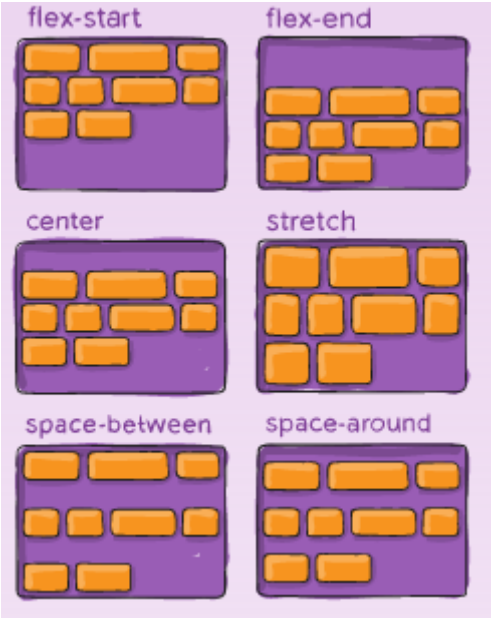
- `stretch` (default): stretch to fill the container (still respect min-width/max-width)
- `flex-start` / `start` / `self-start`: items are placed at the start of the cross axis. The difference between

these is subtle, and is about respecting the `flex-direction` rules or the `writing-mode` rules.

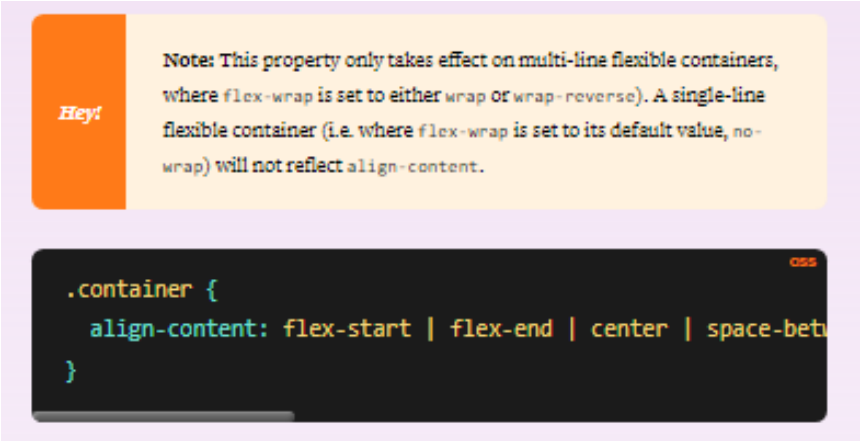
- `flex-end` / `end` / `self-end`: items are placed at the end of the cross axis. The difference again is subtle and is about respecting `flex-direction` rules vs. `writing-mode` rules.
- `center`: items are centered in the cross-axis
- `baseline`: items are aligned such as their baselines align

The `safe` and `unsafe` modifier keywords can be used in conjunction with all the rest of these keywords (although note [browser support]), and deal with helping you prevent aligning elements such that the content becomes inaccessible.

# align-content



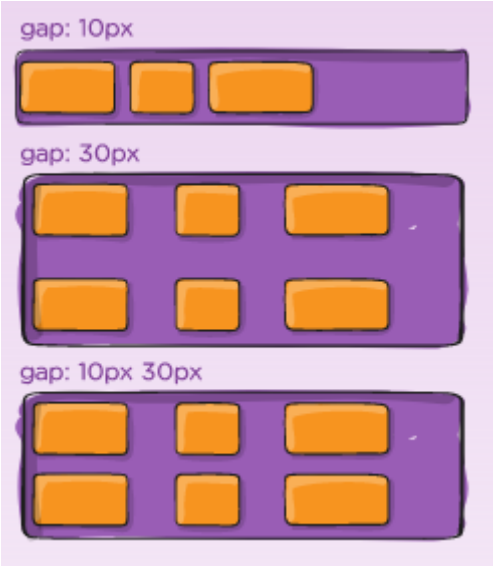
This aligns a flex container's lines within when there is extra space in the cross-axis, similar to how `justify-content` aligns individual items within the main-axis.




- `normal` (default): items are packed in their default position as if no value was set.
- `flex-start` / `start`: items packed to the start of the container. The (more supported) `flex-start` honors the `flex-direction` while `start` honors the `writing-mode` direction.
- `flex-end` / `end`: items packed to the end of the container. The (more support) `flex-end` honors the `flex-direction` while end honors the `writing-mode` direction.
- `center`: items centered in the container
- `space-between`: items evenly distributed; the first line is at the start of the container while the last one is at the end
- `space-around`: items evenly distributed with equal space around each line
- `space-evenly`: items are evenly distributed with equal space around them
- `stretch`: lines stretch to take up the remaining space

The `safe` and `unsafe` modifier keywords can be used in conjunction with all the rest of these keywords (although note browser support), and deal with helping you prevent aligning elements such that the content becomes inaccessible.

# gap, row-gap, column-gap




The `gap` property explicitly controls the space between flex items. It applies that spacing *only between items* not on the outer edges.

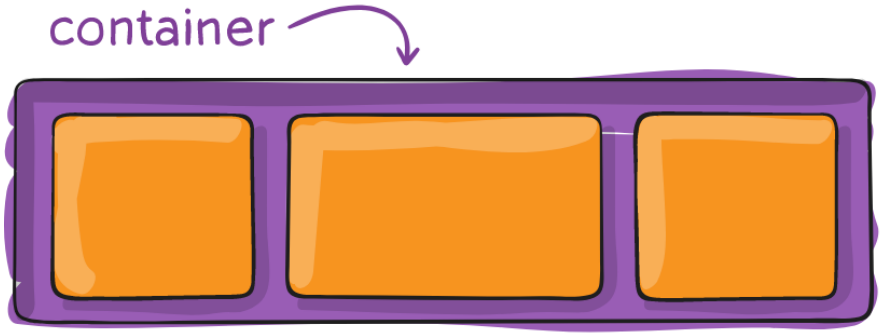
```
.container {
  display: flex;
  ...
  gap: 10px;
  gap: 10px 20px; /* row-gap column gap */
  row-gap: 10px;
  column-gap: 20px;
}
```

The behavior could be thought of as a *minimum* gutter, as if the gutter is bigger somehow (because of something

like `justify-content: space-between;`) then the gap will only take effect if that space would end up smaller.

It is not exclusively for flexbox, `gap` works in grid and multi-column layout as well.

# Flexbox properties




# Properties for the Parent (flex container)

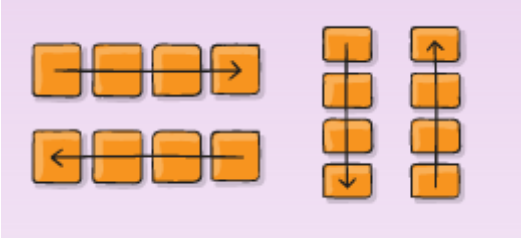
# Display

Isso define um contêiner flexível; inline ou bloco dependendo do valor fornecido. Ele habilita um contexto flexível para todos os seus filhos diretos.

```
.container {
  display: flex; /* or inline-flex */
}
```

Observe que as colunas CSS não têm efeito em um contêiner flexível.

## flex-direction

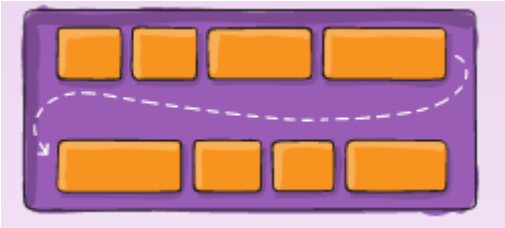


Isso estabelece o eixo principal, definindo assim a direção em que os itens flexíveis são colocados no contêiner flexível. Flexbox é (além da embalagem opcional) um conceito de layout de direção única. Pense nos itens flexíveis como principalmente dispostos em linhas horizontais ou colunas verticais.

```
.container {
  flex-direction: row | row-reverse | column | column-revers
}
```

• linha (padrão): da esquerda para a direita em ltr; direita para esquerda em rtl
• row-reverse: da direita para a esquerda em ltr; da esquerda para a direita em rtl
• coluna: igual à linha, mas de cima para baixo
• reverso da coluna: igual ao reverso da linha, mas de baixo para cima

## flex-wrap



Por padrão, todos os itens flexíveis tentarão caber em uma linha. Você pode alterar isso e permitir que os itens sejam agrupados conforme necessário com essa propriedade.

```
.container {
  flex-wrap: nowrap | wrap | wrap-reverse;
}
```

nowrap (padrão): todos os itens flexíveis estarão em uma linha
wrap: os itens flexíveis serão agrupados em várias linhas, de cima para baixo.

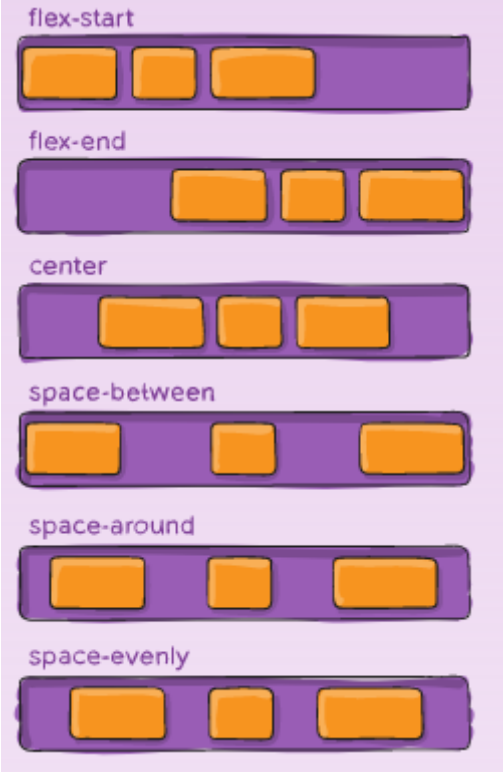
wrap-reverse: os itens flexíveis serão agrupados em várias linhas de baixo para cima.
Existem algumas demonstrações visuais de flex-wrap aqui.

## flex-flow

Esta é uma abreviação para as propriedades flex-direction e flex-wrap, que juntas definem os eixos principal e cruzado do contêiner flex. O valor padrão é row nowrap.

```
.container {
  flex-flow: column wrap;
}
```

# justify-content



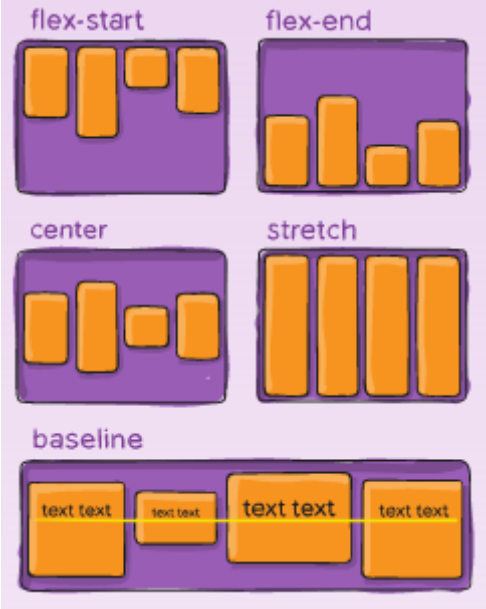

Isso define o alinhamento ao longo do eixo principal. Ele ajuda a distribuir espaço livre extra quando todos os itens flexíveis em uma linha são inflexíveis ou são flexíveis, mas atingiram seu tamanho máximo. Ele também exerce algum controle sobre o alinhamento dos itens quando eles ultrapassam a linha.

```
.container {
  justify-content: flex-start | flex-end | center | space-be
}
```

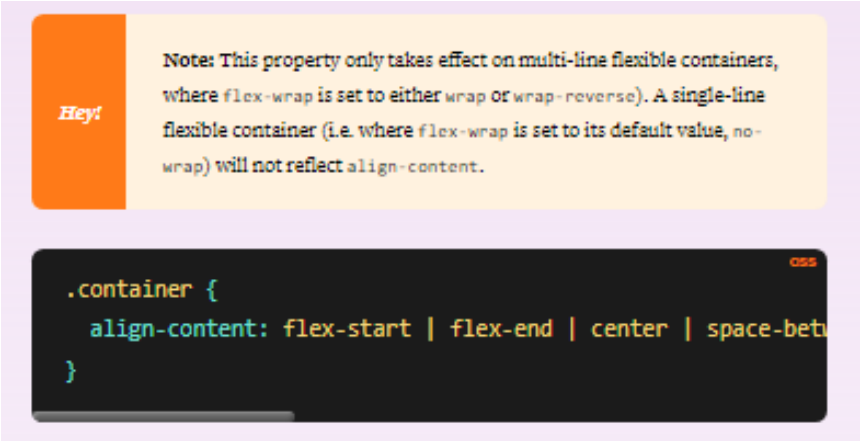
flex-start (padrão): os itens são empacotados no início da flex-direction.
flex-end: os itens são empacotados no final da flex-direction.
start: os itens são empacotados no início da direção do modo de escrita.
end: os itens são empacotados no final da direção do modo de escrita.
left: os itens são empacotados em direção à borda esquerda do contêiner, a menos que isso não faça sentido com o flex-direction, então ele se comporta como start.
right: os itens são empacotados em direção à borda direita do contêiner, a menos que isso não faça sentido com o flex-direction, então ele se comporta como start.
center: os itens são centralizados ao longo da linha
espaço entre: os itens são distribuídos uniformemente na linha; primeiro item está na linha inicial, último item na linha final
space-around: os itens são distribuídos uniformemente na linha com espaço igual ao redor deles. Observe que visualmente os espaços não são iguais, pois todos os itens possuem

espaços iguais em ambos os lados. O primeiro item terá uma unidade de espaço contra a borda do contêiner, mas duas unidades de espaço entre o próximo item porque esse próximo item tem seu próprio espaçamento que se aplica.
space-evenly: os itens são distribuídos de forma que o espaçamento entre quaisquer dois itens (e o espaço até as bordas) seja igual.
Observe que o suporte do navegador para esses valores é diferenciado. Por exemplo, o espaço entre nunca recebeu suporte de algumas versões do Edge e o início/fim/esquerda/direita ainda não está no Chrome. MDN tem gráficos detalhados.
Os valores mais seguros são flex-start, flex-end e center.
Há também duas palavras-chave adicionais que você pode emparelhar com esses valores: safe e unsafe. O uso seguro garante que, independentemente de você fazer esse tipo de posicionamento, você não pode empurrar um elemento de forma que ele seja renderizado fora da tela (por exemplo, fora do topo) de forma que o conteúdo não possa ser rolado também (chamado “perda de dados” ).

# align-items



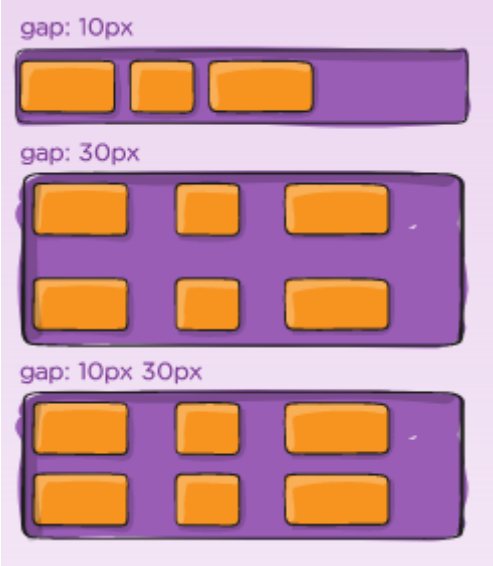

Isso alinha as linhas de um contêiner flexível quando há espaço extra no eixo cruzado, semelhante a como justificar-conteúdo alinha itens individuais no eixo principal.




• normal (padrão): os itens são empacotados em sua posição padrão como se nenhum valor fosse definido.

• flex-start/start: itens embalados até o início do container. O (mais suportado) flex-start respeita a flex-direction enquanto start respeita a direção do modo de escrita.
• flex-end / end: itens embalados até o final do container. O (mais suporte) flex-end respeita a direção flex enquanto end respeita a direção do modo de escrita.
• centro: itens centralizados no contêiner
• espaço entre: itens distribuídos uniformemente; a primeira linha está no início do contêiner enquanto a última está no final
• espaço ao redor: itens distribuídos uniformemente com espaço igual ao redor de cada linha
• espaçar uniformemente: os itens são distribuídos uniformemente com espaço igual ao redor deles
• esticar: as linhas se esticam para ocupar o espaço restante
As palavras-chave do modificador safe e unsafe podem ser usadas em conjunto com todo o restante dessas palavras-chave (embora observe o suporte do navegador) e lidam com a ajuda a evitar o alinhamento de elementos de forma que o conteúdo se torne inacessível.

# gap, row-gap, column-gap



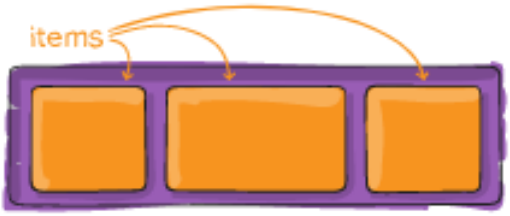

A propriedade gap controla explicitamente o espaço entre os itens flexíveis. Aplica-se esse espaçamento apenas entre os itens e não nas bordas externas.

```
.container {
  display: flex;
  ...
  gap: 10px;
  gap: 10px 20px; /* row-gap column gap */
  row-gap: 10px;
  column-gap: 20px;
}
```

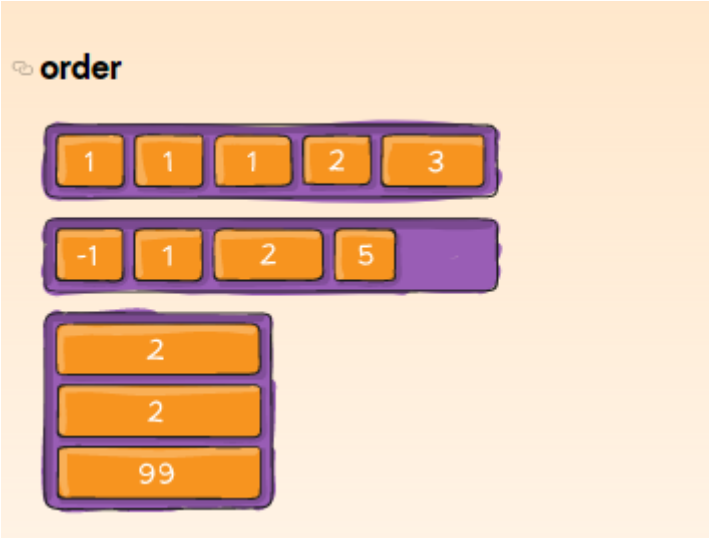
O comportamento pode ser pensado como uma calha mínima, como se a calha fosse maior de alguma forma (por causa de algo como justificar-conteúdo: espaço-entre;) então a lacuna só terá efeito se esse espaço acabar menor.

Não é exclusivamente para flexbox, o gap funciona também em layout de grade e multi-coluna.

# Flexbox properties




**Properties for the Children**
**(flex items)**

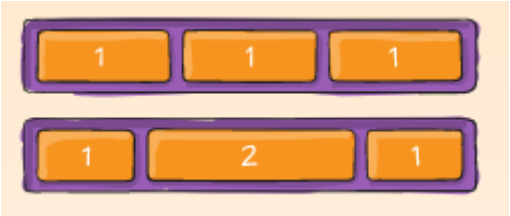



By default, flex items are laid out in the source order. However, the `order` property controls the order in which they appear in the flex container.

```
.item {
  order: 5; /* default is 0 */
}
```

Items with the same `order` revert to source order.

***flex-grow***




This defines the ability for a flex item to grow if necessary. It accepts a unitless value that serves as a proportion. It dictates what amount of the available space inside the flex container the item should take up.

If all items have `flex-grow` set to `1`, the remaining space in the container will be distributed equally to all children. If one of the children has a value of `2`, that child would take up twice as much of the space either one of the others (or it will try, at least).

```
.item {
  flex-grow: 4; /* default 0 */
}
```

Negative numbers are invalid.

***flex-shrink***

This defines the ability for a flex item to shrink if necessary.

```
.item {
  flex-shrink: 3; /* default 1 */
}
```

Negative numbers are invalid.

*flex-basis*

This defines the default size of an element before the remaining space is distributed. It can be a length (e.g. 20%, 5rem, etc.) or a keyword. The `auto` keyword means "look at my width or height property" (which was temporarily done by the `main-size` keyword until deprecated).
The `content` keyword means "size it based on the item's content" – this keyword isn't well supported yet, so it's hard to test and harder to know what its brethren `max-content`, `min-content`, and `fit-content` do.

```
.item {
  flex-basis:  | auto; /* default auto */
}
```

If set to `0`, the extra space around content isn't factored in. If set to `auto`, the extra space is distributed based on its `flex-grow` value. [See this graphic.]

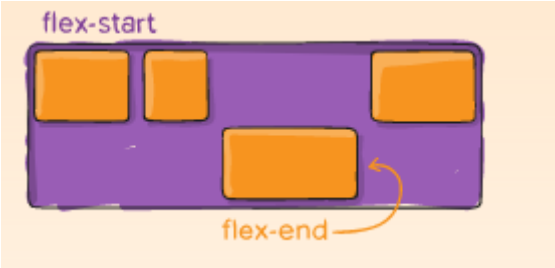
*flex*

This is the shorthand for `flex-grow`, `flex-shrink` and `flex-basis` combined. The second and third parameters (`flex-shrink` and `flex-basis`) are optional. The default is `0 1 auto`, but if you set it with a single number value, like `flex: 5;`, that changes the `flex-basis` to 0%, so it's like setting `flex-grow: 5; flex-shrink: 1; flex-basis: 0%;`.

```
.item {
  flex: none | [ <'flex-grow'> <'flex-shrink'>? || <'flex-bas
}
```

**It is recommended that you use this shorthand property** rather than set the individual properties. The shorthand sets the other values intelligently.

*align-self*



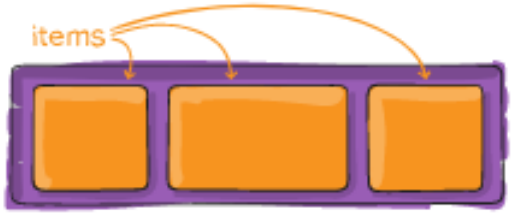

This allows the default alignment (or the one specified by `align-items`) to be overridden for individual flex items.

Please see the `align-items` explanation to understand the available values.

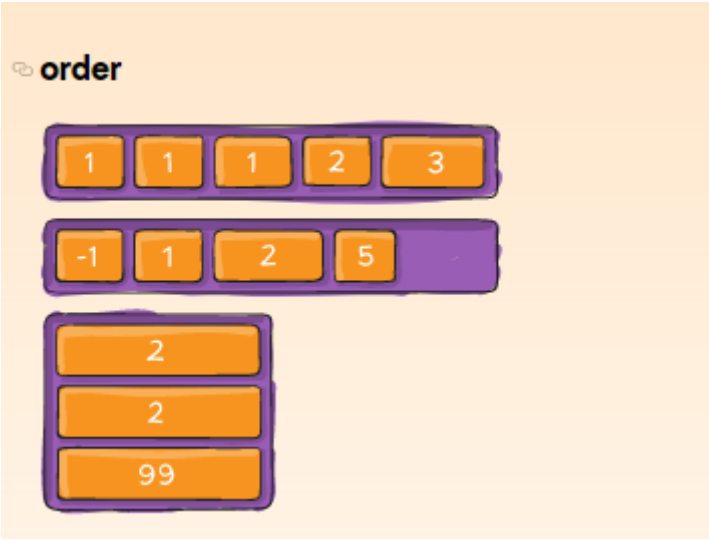
```
.item {
  align-self: auto | flex-start | flex-end | center | basel
}
```

Note that `float`, `clear` and `vertical-align` have no effect on a flex item.

# Flexbox properties




# Properties for the Children (flex items)

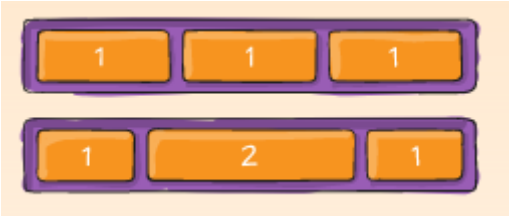



Por padrão, os itens flexíveis são dispostos na ordem de origem. No entanto, a propriedade order controla a ordem em que eles aparecem no contêiner flexível.

```
.item {
  order: 5; /* default is 0 */
}
```

Itens com a mesma ordem revertem para a ordem de origem.

# flex-grow




Isso define a capacidade de um item flexível crescer, se necessário. Aceita um valor sem unidade que serve como proporção. Ele determina a quantidade de espaço disponível dentro do contêiner flexível que o item deve ocupar.
Se todos os itens tiverem flex-grow definido como 1, o espaço restante no contêiner será distribuído igualmente para todos os filhos. Se um dos filhos tiver um valor de 2, essa criança ocupará o dobro do espaço de qualquer uma das outras (ou tentará, pelo menos).

```
.item {
  flex-grow: 4; /* default 0 */
}
```

Números negativos são inválidos.

# flex-shrink

Isso define a capacidade de um item flexível diminuir, se necessário.

```
.item {
  flex-shrink: 3; /* default 1 */
}
```

Números negativos são inválidos.

# flex-basis

Isso define o tamanho padrão de um elemento antes que o espaço restante seja distribuído. Pode ser um comprimento (por exemplo, 20%, 5rem, etc.) ou uma palavra-chave. A palavra-chave auto significa “olhar para minha propriedade largura ou altura” (o que foi temporariamente feito pela palavra-chave main-size até ser descontinuada). A palavra-chave de conteúdo significa “dimensionar com base no conteúdo do item” – essa palavra-chave ainda não é bem suportada, por isso é difícil testar e mais difícil saber o que seus irmãos max-content, min-content e fit-content fazem.

```
.item {
  flex-basis:  | auto; /* default auto */
}
```

Se definido como 0, o espaço extra em torno do conteúdo não é levado em consideração. Se definido como auto, o espaço extra é distribuído com base em seu valor flex-grow. Veja este gráfico.

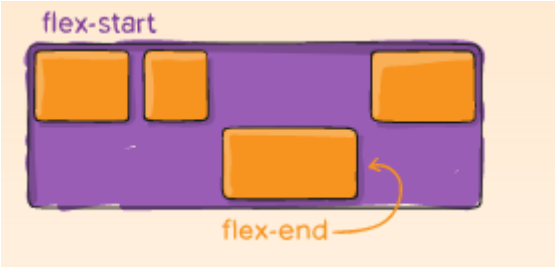
## flex

Esta é a abreviação de flex-grow, flex-shrink e flex-base combinados. O segundo e terceiro parâmetros (flex-shrink e flex-basis) são opcionais. O padrão é 0 1 auto, mas se você definir com um único valor numérico, como flex: 5;, isso altera o flex-basis para 0%, então é como configurar flex-grow: 5; encolhimento flexível: 1; base flexível: 0%;.

```
.item {
  flex: none | [ <'flex-grow'> <'flex-shrink'>? || <'flex-bas
}
```

É recomendável que você use essa propriedade abreviada em vez de definir as propriedades individuais. A taquigrafia define os outros valores de forma inteligente.

## align-self




Isso permite que o alinhamento padrão (ou o especificado por align-items) seja substituído por itens flexíveis individuais.
Por favor, veja a explicação align-items para entender os valores disponíveis.

```
.item {
  align-self: auto | flex-start | flex-end | center | basel
}
```

Observe que float, clear e vertical-align não têm efeito em um item flexível.

# Prefixing Flexbox

Flexbox requires some vendor prefixing to support the most browsers possible. It doesn't just include prepending properties with the vendor prefix, but there are actually entirely different property and value names. This is because the Flexbox spec has changed over time, creating an "old", "tweener", and "new" versions.

Perhaps the best way to handle this is to write in the new (and final) syntax and run your CSS through Autoprefixer, which handles the fallbacks very well.

Alternatively, here's a Sass `@mixin` to help with some of the prefixing, which also gives you an idea of what kind of things need to be done:

```sass
@mixin flexbox() {
  display: -webkit-box;
  display: -moz-box;
  display: -ms-flexbox;
  display: -webkit-flex;
  display: flex;
}

@mixin flex($values) {
  -webkit-box-flex: $values;
  -moz-box-flex:  $values;
  -webkit-flex:  $values;
  -ms-flex:  $values;
  flex:  $values;
}

@mixin order($val) {
  -webkit-box-ordinal-group: $val;
  -moz-box-ordinal-group: $val;
  -ms-flex-order: $val;
  -webkit-order: $val;
  order: $val;
}

.wrapper {
  @include flexbox();
}

.item {
  @include flex(1 200px);
  @include order(2);
}
```

# Prefixing Flexbox

O Flexbox requer alguns prefixos de fornecedores para suportar o maior número possível de navegadores. Ele não inclui apenas propriedades prefixadas com o prefixo do fornecedor, mas na verdade existem nomes de propriedades e valores totalmente diferentes. Isso ocorre porque a especificação do Flexbox mudou ao longo do tempo, criando versões “antigas”, “interpoladas” e “novas”.

Talvez a melhor maneira de lidar com isso seja escrever na nova (e final) sintaxe e executar seu CSS por meio do Autoprefixer, que lida muito bem com os fallbacks.

Alternativamente, aqui está um Sass @mixin para ajudar com alguns prefixos, o que também dá uma ideia de que tipo de coisas precisam ser feitas:

```
@mixin flexbox() {
  display: -webkit-box;
  display: -moz-box;
  display: -ms-flexbox;
  display: -webkit-flex;
  display: flex;
}

@mixin flex($values) {
  -webkit-box-flex: $values;
  -moz-box-flex:  $values;
  -webkit-flex:  $values;
  -ms-flex:  $values;
  flex:  $values;
}

@mixin order($val) {
  -webkit-box-ordinal-group: $val;
  -moz-box-ordinal-group: $val;
  -ms-flex-order: $val;
  -webkit-order: $val;
  order: $val;
}

.wrapper {
  @include flexbox();
}

.item {
  @include flex(1 200px);
  @include order(2);
}
```

# Examples

Let’s start with a very very simple example, solving an almost daily problem: perfect centering. It couldn’t be any simpler if you use flexbox.

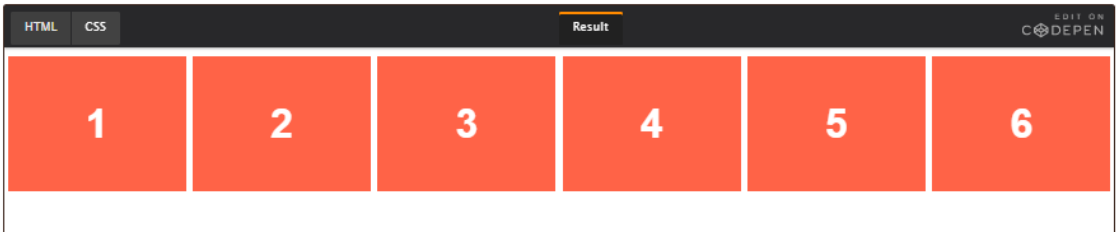
```
.parent {
  display: flex;
  height: 300px; /* Or whatever */
}

.child {
  width: 100px;  /* Or whatever */
  height: 100px; /* Or whatever */
  margin: auto;  /* Magic! */
}
```

This relies on the fact a margin set to `auto` in a flex container absorb extra space. So setting a margin of `auto` will make the item perfectly centered in both axes.

Now let's use some more properties. Consider a list of 6 items, all with fixed dimensions, but can be auto-sized. We want them to be evenly distributed on the horizontal axis so that when we resize the browser, everything scales nicely, and without media queries.

```
.flex-container {
  /* We first create a flex layout context */
  display: flex;

  /* Then we define the flow direction
     and if we allow the items to wrap
   * Remember this is the same as:
   * flex-direction: row;
   * flex-wrap: wrap;
   */
  flex-flow: row wrap;

  /* Then we define how is distributed the remaining space */
  justify-content: space-around;
}
```

Done. Everything else is just some styling concern. Below is a pen featuring this example. Be sure to go to CodePen and try resizing your windows to see what happens.




HTML

```html
<ul class="flex-container">
  <li class="flex-item">1</li>
  <li class="flex-item">2</li>
  <li class="flex-item">3</li>
  <li class="flex-item">4</li>
  <li class="flex-item">5</li>
  <li class="flex-item">6</li>
</ul>
```

CSS

```css
.flex-container {
  /* We first create a flex layout context */
  display: flex;

  /* Then we define the flow direction
     and if we allow the items to wrap
   * Remember this is the same as:
   * flex-direction: row;
   * flex-wrap: wrap;
   */
  flex-flow: row wrap;

  /* Then we define how is distributed the remaining space */
  justify-content: space-around;

  padding: 0;
  margin: 0;
  list-style: none;
}
```

Let’s try something else. Imagine we have a right-aligned navigation element on the very top of our website, but we want it to be centered on medium-sized screens and single-columned on small devices. Easy enough.

```css
/* Large */
.navigation {
  display: flex;
  flex-flow: row wrap;
  /* This aligns items to the end line on main-axis */
  justify-content: flex-end;
}

/* Medium screens */
@media all and (max-width: 800px) {
  .navigation {
    /* When on medium sized screens, we center it by evenly distributing empty space around items */
    justify-content: space-around;
  }
}

/* Small screens */
@media all and (max-width: 500px) {
  .navigation {
    /* On small screens, we are no longer using row direction but column */
    flex-direction: column;
  }
}
```

HTML CSS

```html
<ul class="navigation">
  <li><a href="#">Home</a></li>
  <li><a href="#">About</a></li>
  <li><a href="#">Products</a></li>
  <li><a href="#">Contact</a></li>
</ul>
```

Resources

HTML CSS

```css
.navigation {
  display: flex;
  flex-flow: row wrap;
  justify-content: flex-end;

  list-style: none;
  margin: 0;
  background: deepskyblue;
}
```

Resources

HTML CSS Result EDIT ON CODEPEN

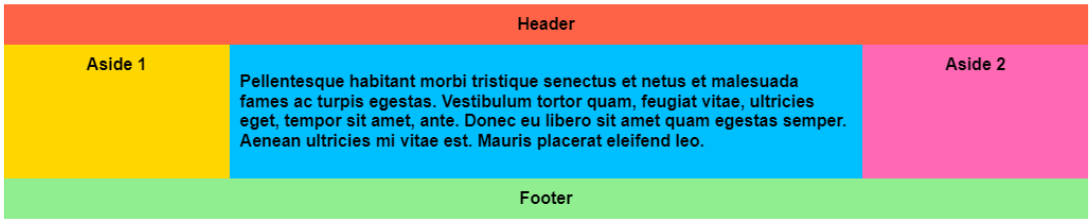
Home About Products Contact

Let’s try something even better by playing with flex items flexibility! What about a mobile-first 3-columns layout with full-width header and footer. And independent from source order.

```
.wrapper {
  display: flex;
  flex-flow: row wrap;
}

/* We tell all items to be 100% width, via flex-basis */
.wrapper > * {
  flex: 1 100%;
}

/* We rely on source order for mobile-first approach
 * in this case:
 * 1. header
 * 2. article
 * 3. aside 1
 * 4. aside 2
 * 5. footer
 */
```

```css
/* Medium screens */
@media all and (min-width: 600px) {
  /* We tell both sidebars to share a row */
  .aside { flex: 1 auto; }
}

/* Large screens */
@media all and (min-width: 800px) {
  /* We invert order of first sidebar and main
   * And tell the main element to take twice as much width as the other two sidebars
   */
  .main { flex: 3 0px; }
  .aside-1 { order: 1; }
  .main    { order: 2; }
  .aside-2 { order: 3; }
  .footer  { order: 4; }
}
```

HTML CSS Result EDIT ON CODEPEN




HTML CSS

```html
<div class="wrapper">
  <header class="header">Header</header>
  <article class="main">
    <p>Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vestibulum tortor quam, feugiat vitae, ultricies eget, tempor sit amet, ante. Donec eu libero sit amet quam egestas semper. Aenean ultricies mi vitae est. Mauris placerat eleifend leo.</p>
  </article>
  <aside class="aside aside-1">Aside 1</aside>
  <aside class="aside aside-2">Aside 2</aside>
  <footer class="footer">Footer</footer>
</div>
```

HTML CSS

```css
.wrapper {
  display: flex;
  flex-flow: row wrap;
  font-weight: bold;
  text-align: center;
}

.wrapper > * {
  padding: 10px;
  flex: 1 100%;
}

.header {
  background: tomato;
}
```

HTML CSS

```css
.footer {
  background: lightgreen;
}

.main {
  text-align: left;
  background: deepskyblue;
}

.aside-1 {
  background: gold;
}

.aside-2 {
  background: hotpink;
```

HTML CSS

```
@media all and (min-width: 600px) {
  .aside { flex: 1 0 0; }
}

@media all and (min-width: 800px) {
  .main    { flex: 3 0px; }
  .aside-1 { order: 1; }
  .main    { order: 2; }
  .aside-2 { order: 3; }
  .footer  { order: 4; }
}

body {
  padding: 2em;
}
```